Student 21 **Lakshmi Narayanan E** 192524249RD 4 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.Scanner;

public class SmartDocumentEditor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Editing history using StringBuffer
        StringBuffer editHistory = new StringBuffer();

        // Step 1: Read paragraph from user
        System.out.println("Enter a paragraph:");
        String paragraph = sc.nextLine();
        editHistory.append("Original Document Recorded.\n");

        // Step 2: Count characters, words, sentences
        int charCount = paragraph.length();
        int wordCount = paragraph.trim().split("\\s+").length;
        String[] sentences = paragraph.split("(?<=[.!?])\\s*");
        int sentenceCount = sentences.length;

        editHistory.append("Counted Characters: ").append(charCount).append("\n");
        editHistory.append("Counted Words: ").append(wordCount).append("\n");
        editHistory.append("Counted Sentences: ").append(sentenceCount).append("\n");

        // Step 3: Replace every occurrence of a given word with another word
        System.out.println("Enter the word to replace:");
        String oldWord = sc.nextLine();
        System.out.println("Enter the new word:");
        String newWord = sc.nextLine();

        String replacedText = paragraph.replaceAll("\\b" + oldWord + "\\b", newWord);
        editHistory.append("Replaced '").append(oldWord).append("' with '")
                    .append(newWord).append("'\n");

        // Step 4: Reverse each sentence individually
        String[] updatedSentences = replacedText.split("(?<=[.!?])\\s*");
        StringBuilder reversedSentences = new StringBuilder();

        for (String sentence : updatedSentences) {
            StringBuilder sb = new StringBuilder(sentence);
            sb.reverse();
            reversedSentences.append(sb.toString()).append(" ");
        }
        editHistory.append("Reversed each sentence individually.\n");

        // Step 6: Construct final formatted document using StringBuilder
        StringBuilder finalDocument = new StringBuilder();
        finalDocument.append("----- Final Formatted Document -----\n");
        finalDocument.append(reversedSentences.toString().trim());
        finalDocument.append("\n-----------------------------------");

        editHistory.append("Final Document Constructed.\n");

        // Step 7: Display everything
        System.out.println("\n===== ORIGINAL DOCUMENT =====");
        System.out.println(paragraph);

        System.out.println("\n===== DOCUMENT STATISTICS =====");
        System.out.println("Characters: " + charCount);
        System.out.println("Words: " + wordCount);
        System.out.println("Sentences: " + sentenceCount);
```

```
        System.out.println("\n===== EDITING HISTORY (StringBuffer) =====");
        System.out.println(editHistory.toString());

        System.out.println("\n===== FINAL DOCUMENT (StringBuilder) =====");
        System.out.println(finalDocument.toString());

        sc.close();
    }
}
```

INPUT

```
This is a test. This is fun! Is this working?
test
demo
```

OUTPUT

```
(no output)
```

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;
public class Main {
    static class Patient {
        int id;
        String name;
        String dept;
        boolean cancelled;
        Patient(int id, String name, String dept) {
            this.id = id;
            this.name = name;
            this.dept = dept;
            this.cancelled = false;
        }
        public String toString() {
            return id + " " + name + " " + dept + (cancelled ? " CANCELLED" : "");
        }
    }
    static List<Patient> patients = new ArrayList<>();
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (sc.hasNext()) {
            String cmd = sc.next();
            if (cmd.equalsIgnoreCase("register")) {
                int id = sc.nextInt();
                String name = sc.next();
                String dept = sc.next();
                register(id, name, dept);
            } else if (cmd.equalsIgnoreCase("display")) {
                displayAll();
            } else if (cmd.equalsIgnoreCase("update")) {
                int id = sc.nextInt();
                String name = sc.next();
                String dept = sc.next();
                updatePatient(id, name, dept);
            } else if (cmd.equalsIgnoreCase("cancel")) {
                int id = sc.nextInt();
                cancelAppointment(id);
            } else if (cmd.equalsIgnoreCase("removeCancelled")) {
                removeCancelled();
            } else if (cmd.equalsIgnoreCase("displayByDept")) {
                String dept = sc.next();
                displayByDept(dept);
            } else if (cmd.equalsIgnoreCase("firstLast")) {
                firstLast();
            }
        }
        sc.close();
    }
    static void register(int id, String name, String dept) {
        patients.add(new Patient(id, name, dept));
    }
    static void displayAll() {
        for (Patient p : patients) {
            if (!p.cancelled) System.out.println(p);
        }
    }
    static void updatePatient(int id, String name, String dept) {
        for (Patient p : patients) {
            if (p.id == id) {
                p.name = name;
                p.dept = dept;
```

```
                return;
            }
        }
    }
    static void cancelAppointment(int id) {
        for (Patient p : patients) {
            if (p.id == id) {
                p.cancelled = true;
                return;
            }
        }
    }
    static void removeCancelled() {
        patients.removeIf(p -> p.cancelled);
    }
    static void displayByDept(String dept) {
        for (Patient p : patients) {
            if (!p.cancelled && p.dept.equalsIgnoreCase(dept)) {
                System.out.println(p);
            }
        }
    }
    static void firstLast() {
        if (patients.isEmpty()) return;
        Patient first = null, last = null;
        for (Patient p : patients) {
            if (!p.cancelled) { first = p; break; }
        }
        for (int i = patients.size() - 1; i >= 0; i--) {
            Patient p = patients.get(i);
            if (!p.cancelled) { last = p; break; }
        }
        if (first != null) System.out.println("First: " + first);
        if (last != null) System.out.println("Last: " + last);
    }
}
```

INPUT

```
register 101 Ravi Cardiology
register 102 Anu Neurology
register 103 Kumar Cardiology
register 104 Priya Orthopedics
display
update 102 Anitha Neurology
cancel 103
removeCancelled
displayByDept Cardiology
firstLast
```

OUTPUT

```
(no output)
```

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;

public class AirportBoardingQueue {
    static class Passenger {
        String name;
        boolean vip;
        int pr;
        Passenger(String n, boolean v, int p) { name = n; vip = v; pr = p; }
        public String toString() { return vip ? name + "(VIP,p=" + pr + ")" : name; }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Queue<Passenger> normal = new LinkedList<>();
        PriorityQueue<Passenger> vip = new PriorityQueue<>(
                (a, b) -> Integer.compare(b.pr, a.pr)
        );
        System.out.print("Enter number of passengers to add: ");
        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            System.out.print("Passenger " + i + ": ");
            String name = sc.next();
            boolean isVip = sc.next().equalsIgnoreCase("Yes");
            int p = sc.nextInt();
            Passenger x = new Passenger(name, isVip, isVip ? p : 0);
            (isVip ? vip : normal).offer(x);
            System.out.println("Added: " + x);
        }
        System.out.print("\nEnter number of boarding operations: ");
        int m = sc.nextInt();
        for (int i = 1; i <= m; i++) {
            if (normal.isEmpty() && vip.isEmpty()) { System.out.println("No passengers remaining."); break; }
            System.out.println("\nOperation " + i);
            Passenger next = !vip.isEmpty() ? vip.peek() : normal.peek();
            System.out.println("Next passenger (without removing): " + next);
            Passenger boarded = !vip.isEmpty() ? vip.poll() : normal.poll();
            System.out.println("Boarded: " + boarded);

            System.out.println("Remaining Normal: " + normal);
            System.out.println("Remaining VIP: " + toSortedList(vip));
        }

        sc.close();
    }

    static List<String> toSortedList(PriorityQueue<Passenger> pq) {
        PriorityQueue<Passenger> copy = new PriorityQueue<>(pq.comparator());
        copy.addAll(pq);
        List<String> out = new ArrayList<>();
        while (!copy.isEmpty()) out.add(copy.poll().toString());
        return out;
    }
}
```

INPUT

```
3
Ravi
Yes
5
Karthik
No
0
Meena
Yes
```

```
8
2
```

OUTPUT

```
(no output)
```

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;
public class CertificateVerification {
    LinkedHashSet<String> certificates = new LinkedHashSet<String>();
    void addCertificate(String id) {
        if (certificates.add(id))
            System.out.println(id + " Added");
        else
            System.out.println(id + " Duplicate Certificate");
    }
    void verifyCertificate(String id) {
        if (certificates.contains(id))
            System.out.println("Certificate Found");
        else
            System.out.println("Certificate Not Found");
    }
    void displayInsertionOrder() {
        System.out.println("Certificates in Insertion Order:");
        for (String id : certificates) {
            System.out.println(id);
        }
    }
    void displaySortedOrder() {
        TreeSet<String> sorted = new TreeSet<String>(certificates);
        System.out.println("Certificates in Sorted Order:");
        for (String id : sorted) {
            System.out.println(id);
        }
    }
    void totalCertificates() {
        System.out.println("Total Unique Certificates: " + certificates.size());
    }
    void removeCertificate(String id) {
        if (certificates.remove(id))
            System.out.println("Certificate Removed");
        else
            System.out.println("Certificate Not Found");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CertificateVerification obj = new CertificateVerification();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            String id = sc.next();
            obj.addCertificate(id);
        }
        obj.displayInsertionOrder();
        String verifyId = sc.next();
        obj.verifyCertificate(verifyId);
        obj.displaySortedOrder();
        obj.totalCertificates();
        String removeId = sc.next();
        obj.removeCertificate(removeId);
        System.out.println("Final Certificate List:");
        obj.displayInsertionOrder();
        sc.close();
    }
}
```

INPUT

```
7
CERT101
CERT205
CERT110
```

```
CERT205
CERT350
CERT450
CERT101
CERT350
CERT110
```

OUTPUT

```
(no output)
```

Q5. Smart Inventory Management using Map Interface

— Not Submitted —